



FACULDADE DE TECNOLOGIA SENAI GASPAR RICARDO JÚNIOR

IFACI: INTERFACE INDUSTRIAL DE ALTA PERFORMANCE PARA GERENCIAMENTO DE DISPOSITIVOS IoT

IFACI: High Performance Industrial Interface for IoT Device Management

Natália Nogueira ¹

RESUMO

Este artigo apresenta o desenvolvimento do IFACI (Interface Industrial de Alta Performance), um sistema full-stack voltado ao monitoramento e controle de dispositivos IoT em ambientes industriais. A solução integra um frontend construído com Next.js, React e TypeScript, uma API RESTful desenvolvida em Node.js com Express, e o Node-RED como plataforma de automação e recebimento de dados de sensores em tempo real. A arquitetura adotada organiza os recursos em quatro entidades principais — usuários, equipamentos, dispositivos e sensores —, todas gerenciadas por meio de operações CRUD expostas via HTTP. O sistema permite o controle remoto de relés de segurança e conexões de dispositivos diretamente pela interface web, além de receber leituras periódicas de temperatura, pressão, umidade, presença e estado de trava provenientes do Node-RED a cada cinco segundos. O trabalho demonstra a viabilidade de construir interfaces industriais de alta performance com tecnologias web modernas, reduzindo a complexidade de implantação sem comprometer a capacidade de supervisão e controle em tempo real.

Palavras-chave: Internet das Coisas; Interface Industrial; Node-RED; REST API; Next.js; ESP32.

ABSTRACT

This paper presents the development of IFACI (High Performance Industrial Interface),

¹Faculdade de Tecnologia SENAI Gaspar Ricardo Júnior, natalianogueir2006@gmail.com

a full-stack system designed for monitoring and controlling IoT devices in industrial environments. The solution integrates a frontend built with Next.js, React, and TypeScript, a RESTful API developed in Node.js with Express, and Node-RED as an automation platform for real-time sensor data ingestion. The adopted architecture organizes resources into four main entities — users, equipment, devices, and sensors — all managed through CRUD operations exposed via HTTP. The system enables remote control of security relays and device connections directly from the web interface, while also receiving periodic readings of temperature, pressure, humidity, presence, and lock state from Node-RED every five seconds. This work demonstrates the feasibility of building high-performance industrial interfaces using modern web technologies, reducing deployment complexity without compromising real-time supervision and control capabilities.

Keywords: Internet of Things; Industrial Interface; Node-RED; REST API; Next.js; ESP32.

1 Introdução

A crescente adoção de dispositivos conectados em ambientes industriais tem impulsionado a demanda por sistemas capazes de centralizar o monitoramento e o controle de múltiplos equipamentos em uma única interface. Esse movimento, amplamente referenciado como Internet das Coisas Industrial (IIoT), representa uma convergência entre redes de comunicação, sensores embarcados e plataformas de software que viabilizam a supervisão em tempo real de processos físicos Silva and Oliveira (2020).

Nesse contexto, interfaces de supervisão e controle — historicamente denominadas HMI (*Human-Machine Interface*) — evoluíram de painéis físicos dedicados para soluções baseadas em tecnologias web, tornando-se mais acessíveis, portáteis e integráveis a infraestruturas de nuvem Almeida and Pereira (2022). A substituição de hardware proprietário por stacks de desenvolvimento abertos reduz custos de implantação e amplia as possibilidades de customização sem abrir mão da capacidade de resposta em tempo real exigida por ambientes industriais.

O presente trabalho descreve o desenvolvimento do IFACI (*Interface Industrial de Alta Performance*), um sistema full-stack construído para a disciplina de Interfaces

Industriais da Faculdade de Tecnologia SENAI Gaspar Ricardo Júnior. O sistema permite gerenciar usuários, equipamentos, dispositivos e sensores IoT por meio de uma interface web responsiva, integrando uma API RESTful em Node.js com o Node-RED para recebimento e notificação de eventos em tempo real. O frontend, desenvolvido em Next.js com TypeScript e Tailwind CSS, atua como o ponto central de interação do operador, enquanto o Node-RED simula o papel de um dispositivo embarcado — análogo a um ESP32 — que publica leituras de sensores periodicamente.

O artigo está organizado da seguinte forma: a Seção 2 apresenta a revisão de literatura sobre IoT industrial e tecnologias empregadas; a Seção 3 descreve a metodologia e a arquitetura do sistema; a Seção 4 detalha os resultados obtidos; e a Seção 5 apresenta as conclusões e perspectivas de trabalhos futuros.

2 Revisão de Literatura

2.1 Internet das Coisas e Aplicações Industriais

A Internet das Coisas pode ser definida como uma infraestrutura global que conecta objetos físicos à rede, permitindo a coleta, o processamento e o intercâmbio de dados sem intervenção humana direta Gubbi et al. (2013). No contexto industrial, essa capacidade se traduz em ganhos expressivos de eficiência operacional: sensores embarcados em máquinas e equipamentos fornecem dados contínuos sobre temperatura, pressão, vibração e outros parâmetros críticos, possibilitando manutenção preditiva e controle de processos em tempo real Silva and Oliveira (2020).

Lee and Lee (2015) destacam que a adoção de IoT em empresas envolve não apenas a escolha de hardware adequado, mas também a definição de plataformas de software capazes de integrar dispositivos heterogêneos e expor seus dados de forma padronizada. Essa necessidade motivou o surgimento de diversas plataformas de middleware IoT, cujas lacunas de interoperabilidade foram sistematicamente analisadas por Mineraud et al. (2016), que identificaram a ausência de APIs unificadas como um dos principais obstáculos à adoção em larga escala.

A integração entre IoT e computação em nuvem, discutida por Botta et al. (2016), amplia ainda mais as possibilidades de processamento e armazenamento de dados gerados por dispositivos embarcados, criando ecossistemas onde a fronteira entre o

mundo físico e o digital se torna cada vez mais tênue.

2.2 Arquitetura REST e Comunicação entre Serviços

O estilo arquitetural REST (*Representational State Transfer*), formalizado por Fielding and Taylor (2002), estabelece um conjunto de restrições para o design de sistemas distribuídos baseados em HTTP. Sua adoção em sistemas IoT é amplamente justificada pela simplicidade de implementação, pela compatibilidade com infraestruturas web existentes e pela facilidade de integração com ferramentas de automação como o Node-RED Luzuriaga et al. (2015).

Em uma arquitetura REST, recursos são identificados por URIs e manipulados por meio dos métodos HTTP padrão — GET, POST, PUT e DELETE —, o que torna a interface da API autodescritiva e facilmente testável. Essa característica é especialmente relevante em ambientes industriais, onde diferentes sistemas precisam consumir os mesmos dados sem acoplamento rígido entre produtor e consumidor.

2.3 Node-RED como Plataforma de Automação IoT

O Node-RED é uma ferramenta de programação visual baseada em fluxos, desenvolvida originalmente pela IBM e atualmente mantida pela OpenJS Foundation Costa and Souza (2021). Sua abordagem de low-code permite conectar dispositivos, APIs e serviços por meio de nós configuráveis interligados em um editor gráfico acessível via navegador. No contexto de IoT industrial, o Node-RED é frequentemente utilizado como camada de integração entre sensores físicos — como os baseados no microcontrolador ESP32 — e sistemas de supervisão, processando e roteando mensagens em tempo real.

O ESP32, desenvolvido pela Espressif Systems, é um microcontrolador de baixo custo com conectividade Wi-Fi e Bluetooth integrados, amplamente adotado em projetos de IoT industriais e acadêmicos Santos and Lima (2019). Sua capacidade de executar código embarcado e se comunicar com servidores remotos via HTTP o torna um candidato natural para o papel de dispositivo de campo em arquiteturas como a do IFACI.

2.4 Tecnologias Web Modernas para Interfaces Industriais

A construção de interfaces industriais com tecnologias web modernas tem se consolidado como uma alternativa viável às soluções proprietárias tradicionais. O Next.js, framework React mantido pela Vercel, oferece renderização híbrida, roteamento baseado em sistema de arquivos e suporte nativo a TypeScript, características que favorecem o desenvolvimento de aplicações robustas e de fácil manutenção Vercel Inc. (2024). O Tailwind CSS, por sua vez, adota uma abordagem utilitária para estilização, permitindo construir interfaces responsivas com alta consistência visual sem a necessidade de folhas de estilo customizadas extensas Tailwind Labs Inc. (2024).

No lado do servidor, o Express.js se destaca como um dos frameworks Node.js mais utilizados para construção de APIs RESTful, oferecendo uma camada mínima de abstração sobre o módulo HTTP nativo e um ecossistema maduro de middlewares OpenJS Foundation (2024). A combinação dessas tecnologias forma um stack coeso que cobre desde a interface do operador até a lógica de negócio e a integração com dispositivos de campo.

3 Metodologia

3.1 Visão Geral da Arquitetura

O IFACI foi concebido como um sistema de três camadas, conforme ilustrado na Figura 1. O frontend, acessível pela porta 3000, é responsável pela interação com o operador e se comunica exclusivamente com a API via requisições HTTP REST. A API, executada na porta 8080, centraliza toda a lógica de negócio, persiste os dados em memória e notifica o Node-RED sobre eventos de CRUD. O Node-RED, disponível na porta 1880, atua simultaneamente como receptor de notificações da API e como publicador de dados de sensores simulados, enviando leituras a cada cinco segundos por meio da rota de atualização de sensor.

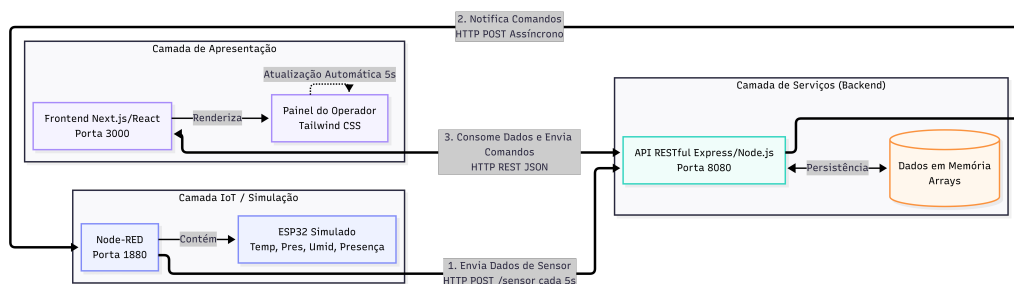


Figura 1: Arquitetura geral do sistema IFACI: frontend, API e Node-RED.

Essa separação de responsabilidades segue os princípios de arquitetura em camadas preconizados pelo estilo REST Fielding and Taylor (2002), garantindo que cada componente possa ser desenvolvido, testado e substituído de forma independente.

3.2 Modelo de Dados

O sistema organiza seus recursos em quatro entidades principais, relacionadas hierarquicamente:

- **Usuários:** representam os operadores do sistema, identificados por nome completo, e-mail e senha. Gerenciados pelas rotas de criação e listagem de usuários.
- **Equipamentos:** agrupam logicamente um conjunto de dispositivos físicos. Cada equipamento possui um identificador numérico incremental e um nome descritivo.
- **Dispositivos:** representam unidades físicas — análogas a microcontroladores ESP32 — vinculadas a um equipamento. Cada dispositivo registra seu status (online/offline), o estado da conexão e o estado do relé de segurança.
- **Sensores IoT:** armazenam leituras de sensores associadas a um dispositivo, incluindo temperatura (°C), pressão (hPa), umidade (%), estado do sensor de presença e estado da trava de segurança.

A Tabela 1 resume os principais endpoints da API e seus respectivos métodos HTTP.

Tabela 1: Principais endpoints da API RESTful do IFACI.

Método	Rota	Descrição
GET	/usuarios	Lista todos os usuários
POST	/novoUsuario	Cria um novo usuário
PUT	/usuarios/:id	Atualiza um usuário
DELETE	/usuarios/:id	Remove um usuário
GET	/equipamentos	Lista todos os equipamentos
POST	/equipamentos	Cria um equipamento
GET	/dispositivos	Lista todos os dispositivos
POST	/dispositivos	Cria um dispositivo
POST	/dispositivos/:id/rele	Libera ou trava o relé
POST	/dispositivos/:id/conexao	Libera ou bloqueia a conexão
GET	/iot	Lista dados de sensores
POST	/sensor	Cria um sensor manualmente
PUT	/sensor/:id	Atualiza sensor (upsert)
DELETE	/sensor/:id	Remove um sensor

3.3 Backend: API RESTful com Node.js e Express

A API foi implementada em Node.js utilizando o framework Express.js OpenJS Foundation (2024). Todos os dados são mantidos em estruturas de array em memória, com identificadores numéricos incrementais gerados pela própria API. Essa abordagem, embora não persistente entre reinicializações, simplifica a implantação e elimina dependências externas de banco de dados, o que é adequado para o escopo acadêmico do projeto.

Um aspecto relevante da implementação é a função de notificação ao Node-RED, que realiza chamadas HTTP assíncronas sempre que um evento de CRUD ocorre em equipamentos, dispositivos ou sensores. A função é executada de forma não bloqueante, garantindo que eventuais indisponibilidades do Node-RED não impactem o tempo de resposta da API. O tratamento de erros é feito com blocos try/catch, registrando avisos no console sem interromper o fluxo principal.

Os endpoints de comando — de controle do relé de segurança e de controle da

conexão — permitem que o operador, diretamente pelo frontend, altere o estado de um dispositivo específico. Após a atualização do estado em memória, a API notifica o Node-RED com o novo estado e um campo de comando que identifica a ação executada, possibilitando que fluxos de automação no Node-RED reajam a esses eventos.

3.4 Frontend: Interface Web com Next.js e TypeScript

O frontend foi desenvolvido com Next.js 16, React 19 e TypeScript, adotando o sistema de roteamento baseado em sistema de arquivos (App Router) introduzido nas versões recentes do framework Vercel Inc. (2024). A estilização utiliza Tailwind CSS 4, que permite compor interfaces responsivas diretamente nas classes dos elementos JSX, sem a necessidade de arquivos CSS separados por componente.

A aplicação é organizada em duas páginas principais:

- **Página de Usuários:** composta pelos componentes de criação e listagem de usuários, permite criar, editar e remover operadores. O componente de listagem expõe uma referência imperativa que permite ao componente pai acionar a atualização da lista após a criação de um novo registro.
- **Página de Equipamentos:** oferece uma visão hierárquica completa do sistema. Cada equipamento exibe seus dispositivos vinculados, e cada dispositivo exibe seus sensores IoT associados, além dos controles de comando.

O componente de listagem de equipamentos implementa um mecanismo de atualização periódica com intervalo de cinco segundos, sincronizado com a frequência de publicação do Node-RED. Isso garante que as leituras de sensores exibidas na interface reflitam o estado mais recente sem exigir interação do operador.

3.5 Integração com Node-RED

O fluxo Node-RED, importado a partir do arquivo de configuração do projeto, é composto por dois grupos funcionais distintos. O primeiro grupo contém um nó de disparo configurado para executar a cada cinco segundos, seguido de um nó de função que gera valores aleatórios para temperatura, pressão, umidade, presença e trava de segurança, e um nó de requisição HTTP que envia esses dados à rota de atualização

de sensor da API. Esse grupo simula o comportamento de um dispositivo ESP32 publicando leituras de sensores em tempo real.

O segundo grupo é composto por doze pares de nós de entrada e resposta HTTP, um para cada evento de CRUD das entidades equipamento, dispositivo e sensor. Cada par recebe a notificação da API, processa o payload em um nó de função que adiciona um timestamp e um campo de evento, e exibe o resultado no painel de depuração do Node-RED. Essa estrutura permite que o Node-RED atue como um barramento de eventos, podendo ser estendido futuramente para acionar alarmes, registrar logs em banco de dados ou encaminhar notificações a sistemas externos.

3.6 Tecnologias Utilizadas

A Tabela 2 consolida as tecnologias empregadas em cada camada do sistema.

Tabela 2: Tecnologias utilizadas por camada no sistema IFACI.

Camada	Tecnologia	Versão
Frontend	Next.js	16.1.7
Frontend	React	19.2.3
Frontend	TypeScript	5.x
Frontend	Tailwind CSS	4.x
Backend	Node.js	18+
Backend	Express.js	5.2.1
Automação IoT	Node-RED	—
Testes de API	Postman	—

4 Resultados e Discussões

4.1 Funcionalidades Implementadas

O sistema IFACI foi implementado com todas as funcionalidades previstas no escopo do projeto. A tela de usuários permite criar, visualizar, editar e remover registros de operadores, com formulários modais para edição que preservam o estado da lista principal. A tela de equipamentos oferece uma visão hierárquica em três níveis —

equipamento, dispositivo e sensor —, com operações CRUD disponíveis em cada nível.

Os controles de comando implementados na interface de dispositivos permitem ao operador:

- **Liberar ou travar o relé de segurança:** o estado atualizado é imediatamente refletido na interface e notificado ao Node-RED, com o campo de trava definido como verdadeiro ou falso conforme a ação do operador.
- **Liberar ou bloquear a conexão com o dispositivo:** o estado de conexão é exibido com indicação visual de cor — azul para ativa, cinza para bloqueada.

Os dados de sensores exibidos na interface incluem temperatura (com duas casas decimais em °C), pressão (com quatro casas decimais em hPa), umidade (com quatro casas decimais em %), estado do sensor de presença e estado da trava de segurança. A atualização automática a cada cinco segundos garante que o operador visualize leituras recentes sem necessidade de recarregar a página.

4.2 Integração e Notificações em Tempo Real

A integração bidirecional entre a API e o Node-RED funcionou conforme projetado. Eventos de criação, edição e exclusão de equipamentos, dispositivos e sensores são notificados ao Node-RED de forma assíncrona, sem impacto no tempo de resposta da API ao cliente. O painel de depuração do Node-RED exibe cada evento com seu timestamp, identificador do recurso e tipo de operação, o que facilita a rastreabilidade de ações realizadas pelo operador.

A publicação periódica de dados pelo Node-RED demonstrou a viabilidade do mecanismo de upsert implementado na API: quando o sensor de identificador 1 não existe, ele é criado automaticamente; nas chamadas subsequentes, seus dados são atualizados. Esse comportamento elimina a necessidade de pré-cadastro manual do sensor e aproxima o sistema do comportamento esperado de um dispositivo ESP32 real que se registra automaticamente ao iniciar.

4.3 Limitações Identificadas

Algumas limitações foram identificadas durante o desenvolvimento e devem ser consideradas em versões futuras do sistema:

- **Persistência em memória:** todos os dados são perdidos ao reiniciar a API. A adoção de um banco de dados — relacional como PostgreSQL ou orientado a documentos como MongoDB — seria necessária para um ambiente de produção.
- **Ausência de autenticação:** os endpoints da API não exigem autenticação, o que representa uma vulnerabilidade em ambientes expostos à rede. A implementação de JWT (JSON Web Tokens) ou OAuth 2.0 seria recomendada.
- **Simulação de sensores:** os dados publicados pelo Node-RED são gerados aleatoriamente, sem representar leituras reais de hardware. A integração com um ESP32 físico ou com um protocolo como MQTT ampliaria a fidelidade do sistema.
- **Ausência de banco de dados de séries temporais:** para aplicações industriais reais, o armazenamento de leituras históricas de sensores em um banco de dados de séries temporais — como InfluxDB — seria essencial para análise de tendências e detecção de anomalias.

4.4 Aplicações Potenciais

A arquitetura do IFACI é aplicável a uma variedade de cenários industriais e acadêmicos. Em ambientes de manufatura, o sistema pode ser adaptado para monitorar linhas de produção, controlando atuadores e coletando dados de sensores distribuídos ao longo do processo. Em instalações prediais, pode gerenciar sistemas de controle de acesso — utilizando o relé de segurança como mecanismo de trava eletrônica — integrado a sensores de presença. Em laboratórios de pesquisa, a arquitetura serve como base para experimentos com protocolos de comunicação IoT, como MQTT e CoAP, substituindo o Node-RED por brokers dedicados Luzuriaga et al. (2015).

A escolha de tecnologias web abertas e amplamente adotadas — Next.js, Express e Node-RED — reduz a barreira de entrada para equipes de desenvolvimento sem experiência prévia em sistemas SCADA ou HMI proprietários, democratizando o acesso a ferramentas de supervisão industrial Almeida and Pereira (2022).

5 Conclusão

Este trabalho apresentou o desenvolvimento do IFACI, uma interface industrial de alta performance construída com tecnologias web modernas para o gerenciamento de dispositivos IoT. O sistema demonstrou que é possível construir uma solução de supervisão e controle funcional, com operações CRUD completas em quatro entidades hierarquicamente relacionadas, integração bidirecional com o Node-RED e controle remoto de atuadores, utilizando exclusivamente ferramentas de código aberto amplamente documentadas.

A arquitetura em três camadas — frontend Next.js, API Express e Node-RED — mostrou-se coesa e de fácil manutenção, com separação clara de responsabilidades entre cada componente. O mecanismo de notificação assíncrona da API ao Node-RED e a atualização periódica do frontend garantiram uma experiência de monitoramento próxima ao tempo real, adequada para o escopo acadêmico do projeto.

Como trabalhos futuros, destacam-se: a substituição do armazenamento em memória por um banco de dados persistente; a implementação de autenticação e autorização nos endpoints da API; a integração com hardware ESP32 real via protocolo MQTT; e a adição de visualizações gráficas de séries temporais para análise histórica das leituras de sensores. Essas evoluções aproximariam o IFACI de uma solução pronta para ambientes industriais reais, mantendo a base tecnológica já estabelecida.

Agradecimentos

Agradeço à instituição de ensino e ao orientador Gabriel que contribuiu para o desenvolvimento deste trabalho.

Referências

- Carlos Almeida and Lucas Pereira. Avaliação de tecnologias web modernas para o desenvolvimento de interfaces homem-máquina (IHM) em ambientes industriais. In *Simpósio Brasileiro de Sistemas de Informação (SBSI)*. Sociedade Brasileira de Computação, 2022.
- Alessio Botta, Walter de Donato, Valerio Persico, and Antonio Pescapé. Integration of cloud computing and Internet of Things: A survey. *Future Generation Computer Systems*, 56:684–700, 2016. doi: 10.1016/j.future.2015.09.021.
- Amanda Costa and Felipe Souza. Desenvolvimento de sistema de supervisão e controle de baixo custo utilizando microcontroladores e a plataforma node-RED. *Revista Brasileira de Mecatrônica e Automação*, 4(2):45–58, 2021.
- Roy T. Fielding and Richard N. Taylor. Principled design of the modern Web architecture. *ACM Transactions on Internet Technology*, 2(2):115–150, 2002. doi: 10.1145/514183.514185.
- Jayavardhana Gubbi, Rajkumar Buyya, Slaven Marusic, and Marimuthu Palaniswami. Internet of things (IoT): A vision, architectural elements, and future directions. *Future Generation Computer Systems*, 29(7):1645–1660, 2013. doi: 10.1016/j.future.2013.01.010.
- In Lee and Kyoochun Lee. The Internet of Things (IoT): Applications, investments, and challenges for enterprises. *Business Horizons*, 58(4):431–440, 2015. doi: 10.1016/j.bushor.2015.03.008.
- Jorge E. Luzuriaga, Miguel Perez, Pablo Boronat, Juan Carlos Cano, Carlos Calafate, and Pietro Manzoni. A comparative evaluation of AMQP and MQTT protocols over unstable and mobile networks. In *2015 12th Annual IEEE Consumer Communications and Networking Conference (CCNC)*, pages 931–936. IEEE, 2015. doi: 10.1109/CCNC.2015.7158101.
- Julien Mineraud, Oleksiy Mazhelis, Xiang Su, and Sasu Tarkoma. A gap analysis of Internet-of-Things platforms. *Computer Communications*, 89–90:5–16, 2016. doi: 10.1016/j.comcom.2016.03.015.

OpenJS Foundation. Express.js documentation, 2024. URL <https://expressjs.com/>.

Bruno Santos and Thiago Lima. Prototipagem de nós sensores para Internet das Coisas utilizando o módulo ESP32: uma abordagem prática. *Revista de Sistemas Embarcados*, 8(1):12–25, 2019.

Roberto Silva and Marcos Oliveira. Aplicações da internet das coisas industrial (IIoT) no monitoramento de processos de manufatura. In *Anais do Congresso Brasileiro de Automática (CBA)*. Sociedade Brasileira de Automática, 2020.

Tailwind Labs Inc. Tailwind CSS documentation, 2024. URL <https://tailwindcss.com/docs>.

Vercel Inc. Next.js documentation, 2024. URL <https://nextjs.org/docs>.